Warp RTG Developer notes

Getting full speed from the Warp's RTG screen memory on the 68060

In short
On a Warp, fast RAM is cached twice (the 68060's own caches and a 96 KB L2 on the board), while VRAM is not cached at all. Writes to VRAM go into a write buffer and are quick; reads are quick on their own; but **switching between reading and writing VRAM costs about 0.6 µs every time**. A drawing loop that reads a VRAM pixel, changes it and writes it back runs at about chip RAM speed, which is how an AGA build of a game can come out faster than its RTG build.

Draw in fast RAM, copy finished pixels to VRAM with long writes or MOVE16, never mix VRAM reads and writes in one loop, and let the OS blit move pixels within VRAM. These rules give the best results on a Warp, and they are good practice on every other RTG system as well.

1. Two kinds of memory in one DDR3

Fast RAM and the RTG screen memory sit in the same DDR3 chips on the Warp board, behind its FPGA. What differs is how the 68060 sees them.

- **Fast RAM is cached.** The 68060 runs it in copyback mode through its 8 KB data cache, and the board adds a 96 KB second-level cache (6-way, 128-byte lines). A load that hits the 68060's cache takes about 20 ns, one that hits the L2 about 100 ns, and one that has to go to the DDR3 600 to 800 ns. Code and data that fit in the L2 run fast; random access across megabytes does not.
- **VRAM is not cached.** The display reads it directly, so the CPU must not keep copies of it: every CPU access to VRAM goes all the way through the FPGA to the DDR3.
- **VRAM writes are buffered.** The FPGA takes a write and lets the CPU go on at once, about 65 ns per write. That cost is per access, whatever its size: a byte write costs as much as a longword write, so writing bytes gets a quarter of the bandwidth. MOVE16, the 68040/68060's 16-byte line transfer, is supported as a burst and is the fastest way in.
- **VRAM reads are not buffered.** A read has to wait for the data, and a read that follows writes must also wait until the write buffer has gone out to the DDR3, so that it sees what was just written. Reads in a row cost about 75 ns each; one read after a run of writes costs about 600 ns. This turnaround, not the reads or the writes themselves, is what makes careless code slow.

2. Measured

Measured with a small benchmark (asm loops, 256 KB per test, best of five) on a Warp with a 68060 at 108 MHz, 640x480x8 screen, 5 October 2026. MB/s; higher is better. The fast RAM tests use 256 KB, more than the L2 holds, so they show streaming from the DDR3.

Access pattern	VRAM	chip RAM	fast RAM
write bytes	15.3	1.8	51.2
write longs	61.5	7.0	75.7
write with MOVE16	77.4	6.2	52.9
read longs	52.2	6.1	85.9
read, modify, write back each long	6.0	2.8	58.8
read 32 bytes, then write them back	19.3	3.2	53.6
write longs, with one read every 64 bytes	39.6	6.5	76.3
copy from fast RAM, long moves	35.6	6.1	40.6
copy from fast RAM, MOVE16	60.5	6.1	44.8
copy to fast RAM, MOVE16	53.2	6.1	–
copy within the same memory, long moves	5.5	2.8	–
copy within the same memory, MOVE16	21.3	3.3	–
BltBitMap within the screen (OS; Warp hardware)	**514**	–	–
WriteChunkyPixels from fast RAM (OS)	35.0	–	–

Read the VRAM column against the others. Plain writes and plain reads are fine, and MOVE16 beats even fast RAM. But a read-modify-write of each pixel group drops to 6 MB/s, close to chip RAM, and a CPU copy from one place in VRAM to another is

the slowest thing on the list, while the OS blitter does the same job almost a hundred times faster. One read slipped into every 64 bytes of writes already costs a third of the write speed.

3. Why an AGA build can beat the RTG build

Chip RAM is slow for everybody: about 7 MB/s for 68060 writes, and slow to read too. A game written for AGA therefore usually draws into fast RAM and converts or copies to chip RAM once a frame (chunky to planar), or lets the blitter work while the CPU does something else. Ported to RTG, the same game is often changed to draw straight into the locked screen bitmap, and that is where it goes wrong. On a Warp, the typical drawing operations that touch the destination become read-modify-write on VRAM:

- masked or transparent sprites and fonts that read the background pixel first,
- blending, shading and XOR effects done in place,
- scrolling or window moves done by copying VRAM to VRAM with the CPU,
- collision or "what colour is here" tests that read the screen,
- byte-sized pixel writes, which cost as much each as longword writes.

Each of these pays the 0.6 µs turnaround over and over, and the screen update ends up at chip RAM speed or below, with no blitter running alongside. Other RTG boards (Zorro II/III cards, other accelerators' RTG) reach VRAM over slower buses, so everything is slower there, but the step from "write" to "read-modify-write" is usually relatively smaller. Code that works acceptably there can still lose most of the Warp's VRAM speed, because the Warp's fast write path is exactly what the turnaround throws away.

4. What to do

- **Keep a frame (or the changing part of it) in fast RAM and draw there.** Sprites, masks, blending and tests all read and write cached memory at full speed. Copy the finished pixels to VRAM once per frame, and only the rows or rectangles that changed.
- **Don't mix VRAM reads and writes in one loop.** Reading VRAM on its own is fine (52 MB/s for long reads, 53 MB/s with MOVE16 into fast RAM); what costs is every read that follows a write. If something on screen is needed, read it first, in a block, into fast RAM, and work on the copy. If a read-modify-write on VRAM cannot be avoided, read a block into registers, change it and write it back: 32 bytes at a time is three times faster than going long by long. And since VRAM is not cached, data read more than once is better kept in fast RAM, where the second read is nearly free.
- **Write wide.** Longwords at least; MOVE16 where you can. Never write pixel by pixel in bytes.
- **Move pixels within VRAM with the OS.** On 8-bit and 16-bit screens, plain copies with BltBitMap, BltBitMapRastPort or ScrollRaster are done by the Warp's hardware: 514 MB/s, against 5.5 MB/s for a CPU loop. (In 32-bit modes, and for blits with other minterms, P96 does the work with the CPU.)
- **Use the screen's own pixel format.** The Warp offers CLUT (8-bit), R5G6B5 (16-bit, big-endian) and B8G8R8A8 (32-bit). Render in that format so the copy is a plain copy, without conversion.
- **Lock briefly.** p96LockBitMap() (or LockBitMapTags()) gives the address and BytesPerRow; copy and unlock straight away. WriteChunkyPixels / p96WritePixelArray are a reasonable fallback (35 MB/s) when locking is not possible.

5. A frame copy with MOVE16

MOVE16 transfers one 16-byte line and ignores the low four address bits, so both source and destination rows must be 16-byte aligned and a whole number of lines long. Allocate the fast RAM buffer with 16 bytes to spare and align it yourself (AllocVec only promises 8). Check the screen's row address and BytesPerRow, and fall back to CopyMemQuick or long moves when they are not aligned, or on a 68020/68030, which has no MOVE16.

```
; void CopyRows16(UBYTE *src, UBYTE *dst, ULONG bytesPerRow, ULONG rows, ULONG dstModulo)
; src rows packed, dst rows dstModulo apart; bytesPerRow a multiple of 64,
; src, dst and dstModulo multiples of 16
_CopyRows16
    movem.l d2-d3/a2,-(sp)
    move.l  16(sp),a0               ; src
    move.l  20(sp),a1               ; dst
    move.l  24(sp),d0               ; bytesPerRow
    move.l  28(sp),d1               ; rows
    move.l  32(sp),d2               ; dstModulo
    lsr.l   #6,d0                   ; 64 bytes per pass
.row
    move.l  a1,a2
    move.l  d0,d3
.line
    move16  (a0)+,(a2)+
    move16  (a0)+,(a2)+
    move16  (a0)+,(a2)+
    move16  (a0)+,(a2)+
    subq.l  #1,d3
    bne.s   .line
    add.l   d2,a1                   ; next screen row
    subq.l  #1,d1
    bne.s   .row
    movem.l (sp)+,d2-d3/a2
    rts
```

C side: lock with p96LockBitMap(bm, &ri, sizeof(ri)), check ri.RGBFormat and that ri.Memory and ri.BytesPerRow are multiples of 16, call CopyRows16(buffer, ri.Memory + y * ri.BytesPerRow, width * bytesPerPixel, rows, ri.BytesPerRow), then p96UnlockBitMap(). Measured on a Warp: 60 MB/s from fast RAM, against 36 MB/s for a long-move copy and 35 MB/s for WriteChunkyPixels.

6. On other systems

Nothing here is Warp-only. Every RTG card keeps its VRAM uncached and behind a bus that is slower to read than to write, so a fast RAM back buffer, wide or burst writes, no loops that mix VRAM reads with writes, and OS blits within VRAM are the right design everywhere; the Warp just rewards them most, and punishes their absence most. Two portability points: MOVE16 exists only on the 68040 and 68060, so test for it and keep a CopyMemQuick or long-move path; and on cards or buses without burst support MOVE16 still works (the transfer is split into longwords) but gains nothing, so there the long-move copy is just as good.

Checklist

- Drawing happens in a fast RAM buffer, not in the locked screen bitmap.
- No loop both reads and writes VRAM. Screen data that is needed is read first, in a block.
- VRAM is written in longwords or with MOVE16, never pixel by pixel in bytes.
- Only changed rows or rectangles are copied, once per frame.
- Moves within VRAM (scrolling, windows) use BltBitMap or ScrollRaster.
- The buffer is in the screen's own pixel format, 16-byte aligned, with a 16-byte multiple per row.
- MOVE16 only on a 68040/68060 and only when aligned; CopyMemQuick or long moves otherwise.
- The bitmap lock is held only for the copy.

The measurements were taken with RTGBench (68060 asm loops timed with the E-clock). Results may vary with the board model, the CPU clock and the screen mode.